

## GW5 (FSDS): Hashing Methods

### Activity 1:

Comparison of Tree Data Structures and Indexing Methods:

| Aspect                    | Static Hashing                                                                                              | Dynamic Hashing                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Structure</b>          | The size of the hash table is fixed at initialization and cannot be changed.                                | The size of the hash table can grow or shrink dynamically based on the requirements.                    |
| <b>Collision Handling</b> | Often managed using techniques like chaining.                                                               | Similar techniques are used, but resizing the table helps distribute keys better and reduce collisions. |
| <b>Efficiency</b>         | Efficient as long as the table is not overloaded.                                                           | More efficient for variable or unpredictable data volumes, as the table adjusts dynamically.            |
| <b>Resizing Cost</b>      | Not applicable since the table size is fixed.                                                               | Resizing involves rehashing existing data, which can be computationally expensive.                      |
| <b>Memory Usage</b>       | May lead to over-allocation (if too large) or degraded performance (if too small).                          | Adapts memory usage dynamically, often leading to more efficient utilization over time.                 |
| <b>Synonyms</b>           | It is possible to find records that do not belong to the block (e.g., in simple cases like linear probing). | Blocks always contain synonyms.                                                                         |
| <b>Use Case</b>           | Suitable for datasets with a fixed size or a known maximum size.                                            | Ideal for applications with unknown or frequently changing data sizes.                                  |

Table 1: Comparison of Static Hashing and Dynamic Hashing

### Activity 12:

1) File structure:

```
Tblock = structure
    T: array[b] of rec; // array of records (fixed format)
    nb: integer; // number of records in T (between 0 and b)
end;
F: file of Tblock buffer buf header (int nb_blocks (N), int LastBlock);
```

2) Search algorithm 1.

---

**Algorithm 1** Search Algorithm

---

```
1: procedure SEARCH(c: integer, var found: boolean, var record: Trec)
2:   Variables: i, j, N, LastBlock: integer
3:   open(F)
4:   N  $\leftarrow$  header(F, 1)
5:   LastBlock  $\leftarrow$  header(F, 2)
6:   i  $\leftarrow$  h(c) // i = (c mod N) + 1
7:   readBlock(F, i, buf)
8:   j  $\leftarrow$  1
9:   found  $\leftarrow$  FALSE
10:  while not found and j  $\leq$  buf.nb do
11:    if buf.T[j].key = c then
12:      found  $\leftarrow$  TRUE
13:      record  $\leftarrow$  buf.T[j]
14:    else
15:      j  $\leftarrow$  j + 1
16:    end if
17:  end while
18:  if not found and buf.nb = b then
19:    i  $\leftarrow$  N + 1
20:    while not found and i  $\leq$  LastBlock do
21:      readBlock(F, i, buffer)
22:      j  $\leftarrow$  1
23:      while not found and j  $\leq$  buf.nb do
24:        if buf.T[j].key = c then
25:          found  $\leftarrow$  TRUE
26:          record  $\leftarrow$  buf.T[j]
27:        else
28:          j  $\leftarrow$  j + 1
29:        end if
30:      end while
31:      if not found and buf.nb = b then
32:        i  $\leftarrow$  i + 1
33:      end if
34:    end while
35:  end if
36:  close(F)
37: end procedure
```

---

## Activity 7:

1) File structure:

```
Tblock1 = structure
  T: array[b1] of rec; // array of records (fixed format)
  nb: integer; // number of records in T (between 0 and b1)
  block_link: integer //Position of the first overflow block
  pos_link: integer //Position of the first record in the overflow block

end;
```

```

Tblock2 = structure
    T: array[b2] of rec_flow; // array of records (fixed format)
    nb: integer; // number of records in T (between 0 and b2)

end;

rec_flow = structure
    rec_val: rec;
    block_link: integer //Position of the next overflow block
    pos_link: integer //Position of the next record in the overflow block
end;

FP: file of Tblock1 buffer buf header (int nb\_blocks, int p (number of the block to be split),
int i (hash function parameter));

F0: file of Tblock2 buffer buf2 header (int nb\_blocks);

```

- 2) Search algorithm 2.
- 3) We store the address of the first empty cell in the overflow zone within the header of the overflow file. Whenever a record is deleted, its location is added to the list of deleted (empty) records.

## Activity 10:

- 1) Les déclarations:

```

const int maxp = P; // Maximum size of primary blocks
const int maxd = D; // Maximum size of overflow blocks

typedef struct Tblocp {
    array T[maxp]; // Array of records
    int nb;        // Number of elements
    int link;      // Link to next block
} Tblocp;

typedef struct Tblocd {
    array T[maxd]; // Array of records
    int nb;        // Number of elements
    int link;      // Link to the overflow zone head block
} Tblocd;

typedef struct Headerp {
    int adrLastBloc; // Address of the last block
    int nbRec;       // Number of records
    int i;           // Level of the file (hash function)
    int p;           // Current block (hash function)
}

```

```

} Headerp;

typedef struct Headerd {
    int adrLastBloc; // Address of the last block
    int nbRec;       // Number of records
} Headerd;

typedef struct TnOF {
    FILE *filep; // Primary file
    Headerp headerp; // Header for the primary file
} TObF;

typedef struct LnOF {
    FILE *filerd; // Overflow file
    Headerd headerd; // Header for the overflow file
} LObF;

Tblocp bufp; // Buffer for primary blocks
Tblocd bufd; // Buffer for overflow blocks

```

### **Algorithm: Insertion Procedure**

2) You must order the primary zone to use the binary search.

---

**Algorithm 2** Search Procedure

---

```
1: procedure SEARCH( $x$ : KeyType, var found: boolean, var ii, jj: integer, var overflow: boolean)
2:    $p \leftarrow \text{getHeader}(FP, 2)$ 
3:    $i \leftarrow \text{getHeader}(FP, 3)$ 
4:    $ii \leftarrow h(i, x)$  //Dynamic hash function
5:   if  $ii < p$  then
6:      $ii \leftarrow h(i + 1, x)$ 
7:   end if
8:    $jj \leftarrow 1$ 
9:    $found \leftarrow \text{FALSE}$ 
10:   $overflow \leftarrow \text{FALSE}$  //Sequential search in the main block
11:  ReadBlock( $FP, ii, Buf$ )
12:  while not  $found$  and  $jj \leq \text{Buf.nb}$  do
13:    if  $Buf.T[jj].key = x$  then
14:       $found \leftarrow \text{TRUE}$ 
15:    else
16:       $jj \leftarrow jj + 1$ 
17:    end if
18:  end while
19:  if not  $found$  and  $Buf.block\_link > -1$  then //Search in the overflow block if the record
    is not found
20:     $ii \leftarrow Buf.block\_link$ 
21:     $jj \leftarrow Buf.pos\_link$ 
22:     $exit \leftarrow \text{FALSE}$ 
23:    while not  $exit$  do
24:      ReadBlock( $FO, ii, buf2$ )
25:      if  $buf2.T[jj].rec\_val.key = x$  then
26:         $found \leftarrow \text{TRUE}$ 
27:         $exit \leftarrow \text{TRUE}$ 
28:      else //Keep searching in the chained list of blocks
29:        if  $buf2.T[jj].block\_link > -1$  then
30:           $ii \leftarrow buf2.T[jj].block\_link$  //The block of the next record
31:           $jj \leftarrow buf2.T[jj].pos\_link$  //The position of the next record
32:        else //Find where to insert the new record cause we couldn't find x
33:           $exit \leftarrow \text{TRUE}$ 
34:          if  $buf2.nb < b2$  then //If the current block is not full(it is the last block of
    the file)
35:             $jj \leftarrow buf2.nb + 1$ 
36:          else
37:             $ii \leftarrow \text{getHeader}(FO, 1)$ 
38:            ReadBlock( $FO, ii, buf2$ )
39:            if  $buf2.nb < b2$  then //Insert the new record in the last position of the
    last block if it's not full.
40:               $jj \leftarrow buf2.nb + 1$ 
41:            else //Allocate a new block
42:               $ii \leftarrow ii + 1$ 
43:               $jj \leftarrow 1$ 
44:            end if
45:          end if
46:        end if
47:      end if
48:    end while
49:  end if
50: end procedure
```

---

---

**Algorithm 3** Search Procedure

---

```
1: procedure SEARCH(key: int, var i, j, ip: int, var found, overflow: bool)
2:    $found \leftarrow \text{false}$ ,  $overflow \leftarrow \text{false}$ 
3:    $i \leftarrow h_1(\text{key})$ 
4:   if  $i < \text{Headerp.p}$  then
5:      $i \leftarrow h_2(\text{key})$ 
6:   end if
7:   readBlock( $filep, i, bufp$ )
8:   Sequential search for key in  $bufp$ 
9:   if not  $found$  then
10:    if  $bufp.link \neq -1$  then
11:       $ip \leftarrow i$ 
12:       $overflow \leftarrow \text{true}$ 
13:       $i \leftarrow bufp.link$ 
14:      readBlock( $filed, i, bufd$ )
15:      Sequential search in  $bufd$  and the associated block list.
16:    end if
17:  end if
18: end procedure
```

---

---

**Algorithm 4** Insertion Procedure

---

```
1: procedure INSERT(key: int)
2:   SEARCH( $i, j, ip, found, overflow$ )
3:   if not  $found$  then
4:     if not  $overflow$  then
5:       readBlock( $filep, i, bufp$ )
6:       if  $bufp.nb < maxp$  then
7:         Insert key into  $bufp.T[j]$ 
8:          $bufp.nb \leftarrow bufp.nb + 1$ 
9:         Write updated  $bufp$  back to  $filep$ 
10:      else
11:        Allocate new overflow block  $i_2$ 
12:        Initialize  $bufd$  with key and set  $bufd.link \leftarrow -1$ 
13:        Update  $bufp.link \leftarrow i_2$ 
14:        Write  $bufp$  to  $filep$  and  $bufd$  to  $filed$  to save the links.
15:      end if
16:    else
17:      readBlock( $filed, i, bufd$ )
18:      if  $bufd.nb < maxd$  then
19:        Insert key into  $bufd.T[bufd.nb]$ 
20:         $bufd.nb \leftarrow bufd.nb + 1$ 
21:        Write updated  $bufd$  back to  $filed$ 
22:      else
23:        Allocate new overflow block  $i_2$ 
24:        Initialize  $bufd2$  with key and link  $bufd2.link \leftarrow bufd.link$ 
25:        Update  $bufp.link \leftarrow i_2$ 
26:        Write  $bufd2$  to  $filed$ 
27:        Write updated  $bufp$  to  $filep$  using  $ip$ 
28:      end if
29:    end if
30:    if splitting condition is met then
31:      Allocate a new block in  $filep$ 
32:      Split block  $p$  and its overflow area
33:      if  $p = 2^i \times n$  then
34:         $p \leftarrow 0$ 
35:         $i \leftarrow i + 1$ 
36:      end if
37:      Update header variables  $headerp.i$  and  $headerp.p$ 
38:    end if
39:  end if
40: end procedure
```

---